

Árvore Binária de Busca - Relatório Técnico

Análise de Desempenho - Atividade 01

Atividade de Estrutura de Dados 2

Aluno: Vinicius da Silva Nunes

viniciusnunes02612@gmail.com

16 de setembro de 2026

Resumo

Este relatório apresenta a implementação e a avaliação de uma árvore binária de busca (ABB) genérica em linguagem C. A estrutura é utilizada para armazenar registros de clientes, indexados por CPF, e registros de cidades, indexados por CEP. São descritas as estruturas de dados, as operações de inserção, busca e remoção, os problemas encontrados durante o desenvolvimento e os resultados de um experimento com 100 CPFs inseridos em quatro ordens diferentes.

Sumário

1	Introdução	2
2	Sessões específicas	2
2.1	Árvore binária de busca	2
2.2	Estruturas de registros	3
2.3	Fluxograma: cadastrar o endereço de um cliente	4
2.4	Operações principais	4
2.5	Validação dos dados	6
3	Problemas	6
4	Resultados	7
4.1	Equipamento utilizado	7
4.2	Experimento com 100 CPFs	7
5	Discussões	8
6	Conclusão	8

1 Introdução

Árvores binárias de busca são estruturas de dados destinadas a organizar elementos de acordo com uma chave. Em uma árvore desse tipo, as chaves menores que a chave de um nó são armazenadas na subárvore esquerda, enquanto as maiores ficam na subárvore direita. Essa organização permite realizar buscas sem percorrer necessariamente todos os elementos.

O objetivo deste trabalho foi implementar, em C, uma ABB capaz de trabalhar com dois tipos de registro: clientes e cidades. O CPF é utilizado como chave dos clientes e o CEP como chave das cidades. A solução adotada utiliza um nó genérico, formado por uma união dos registros e por uma enumeração que identifica qual tipo de dado está armazenado.

Além das operações básicas, foi realizado um experimento com 100 CPFs válidos. Os mesmos dados foram inseridos em quatro ordens: crescente, decrescente, com o elemento central inserido primeiro e totalmente aleatória. O número de nós visitados nas buscas e a altura das árvores foram utilizados para analisar o impacto da ordem de inserção no desempenho.

2 Sessões específicas

2.1 Árvore binária de busca

Uma ABB é composta por nós ligados por ponteiros. Cada nó possui uma chave, um filho à esquerda e um filho à direita. A propriedade fundamental da estrutura é:

$$\text{chaves da esquerda} < \text{chave do nó} < \text{chaves da direita.} \quad (1)$$

No programa, a estrutura principal é definida como `ArvoreBinaria`. Ela contém o campo `tipoN`, que informa se o nó armazena um cliente ou uma cidade, o campo `info`, que é uma união dos dois registros, e os ponteiros `esquerda` e `direita`. Assim, o mesmo algoritmo de árvore pode ser usado nas duas situações.

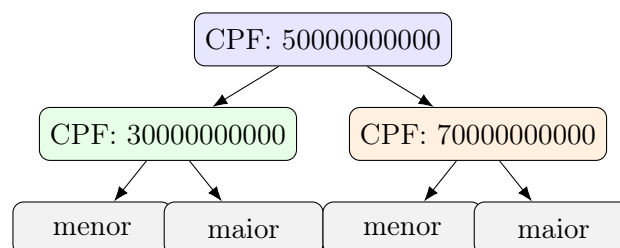


Figura 1: Exemplo de árvore binária de busca organizada por CPF.

A função `pegaChave` abstrai a diferença entre os registros. Quando o nó é do tipo cliente, ela retorna o CPF; quando é do tipo cidade, retorna o CEP. Como as chaves são armazenadas como strings de tamanho fixo, a comparação com `strcmp` preserva a ordenação dos valores.

2.2 Estruturas de registros

O registro de cliente contém CPF, nome, ano de nascimento, telefone, gênero e endereço. O registro de cidade contém CEP, nome, estado e endereço. O endereço é armazenado diretamente dentro dos registros, como composição, e não como um ponteiro separado.

A união `ContemCidadeECliente` permite que o nó ocupe espaço para apenas um dos registros por vez. O campo `tipoN` funciona como uma etiqueta que indica qual membro da união está válido. Essa decisão reduz a duplicação do algoritmo, pois inserção, busca, remoção e impressão podem ser implementadas de forma comum.

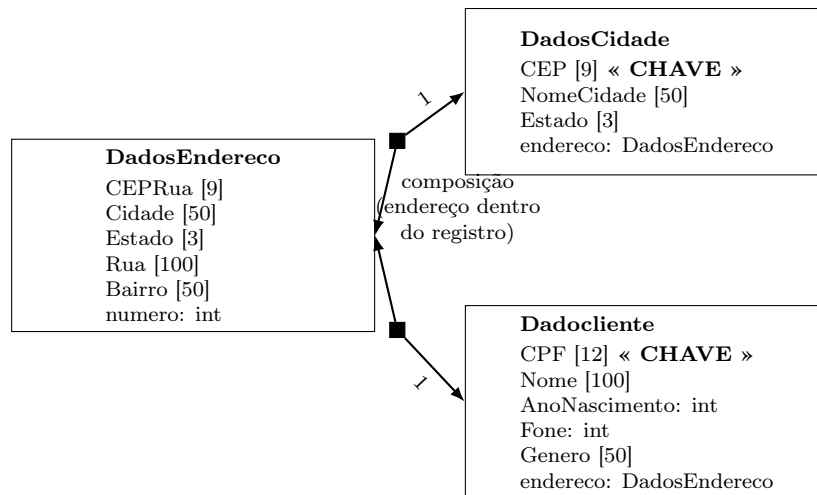


Figura 2: Estruturas de dados e relações de composição entre endereço, cidade e cliente.

A figura 3 apresenta como o nó genérico utiliza a etiqueta `tipoN` para identificar o registro armazenado na `union`. Os ponteiros `esquerda` e `direita` conectam o nó às subárvores, permitindo que a mesma estrutura represente clientes e cidades.

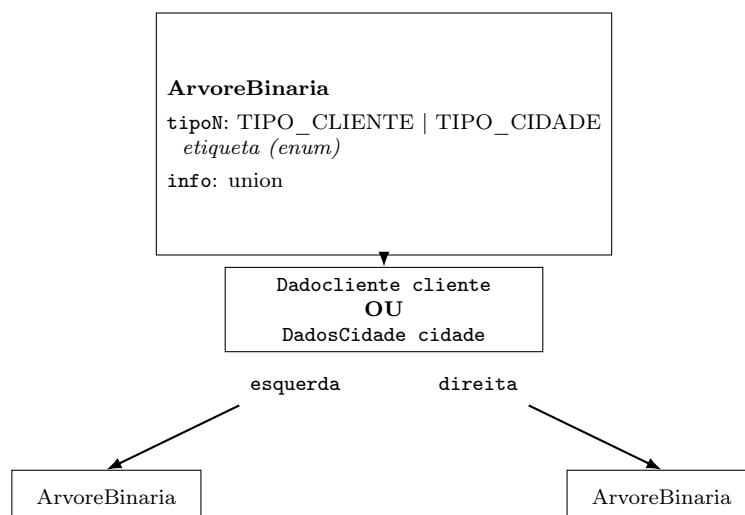


Figura 3: Nó genérico da árvore binária com união, etiqueta de tipo e ponteiros para os filhos.

2.3 Fluxograma: cadastrar o endereço de um cliente

O fluxograma a seguir representa o processo de localização do cliente, leitura e validação do endereço e atualização do registro encontrado.

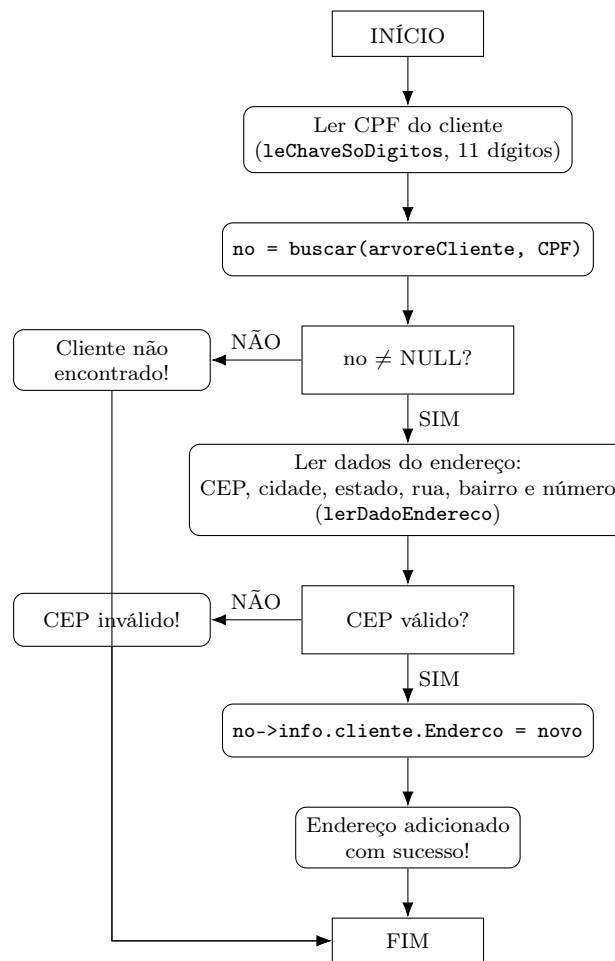


Figura 4: Fluxograma do cadastro do endereço de um cliente.

2.4 Operações principais

- **Inserção:** compara a chave do novo nó com a chave da raiz e percorre recursivamente a subárvore esquerda ou direita. Uma chave repetida não é inserida.
- **Busca:** segue o mesmo caminho determinado pelas comparações. Retorna o endereço do nó encontrado ou NULL quando a chave não existe.
- **Remoção:** trata três casos: nó folha, nó com um filho e nó com dois filhos. No último caso, o nó é substituído pelo menor elemento da subárvore direita.
- **Percurso e liberação:** a árvore pode ser percorrida para listar os registros e liberada recursivamente ao final do programa.

As funções abaixo são implementadas principalmente em `respostas.c` e trabalham com as árvores de clientes e de cidades:

`ArvoreBinaria *CriaArvore()` não recebe parâmetros e retorna uma árvore vazia (NULL), usada para inicializar a raiz.

`ArvoreBinaria *criaNoCliente(struct Dadocliente cliente)` recebe os dados de um cliente, cria e inicializa um nó do tipo cliente; retorna o endereço do novo nó ou NULL se não houver memória disponível.

`ArvoreBinaria *criaNoCidade(struct DadosCidade cidade)` recebe os dados de uma cidade, cria e inicializa um nó do tipo cidade; retorna o endereço do novo nó ou NULL se não houver memória disponível.

`char *pegaChave(ArvoreBinaria *no)` recebe um nó e retorna sua chave: o CPF, quando é um cliente, ou o CEP, quando é uma cidade.

`int inserirArvore(ArvoreBinaria **raiz, ArvoreBinaria *NovoNo)` recebe o endereço da raiz e o nó a ser inserido. Compara as chaves e insere recursivamente na posição correta da árvore; retorna 1 quando insere e 0 quando a chave já existe ou o nó é inválido.

`ArvoreBinaria *buscar(ArvoreBinaria *raiz, char *chave)` recebe a raiz e uma chave, percorre a árvore comparando os valores e retorna o endereço do nó encontrado ou NULL caso a chave não exista.

`ArvoreBinaria *buscarComPassos(ArvoreBinaria *raiz, char *chave, int *passos)` realiza a mesma busca, atualizando a quantidade de nós visitados; retorna o nó encontrado ou NULL. É utilizada no experimento de desempenho.

`int removerNo(ArvoreBinaria **raiz, char *chave)` recebe o endereço da raiz e a chave do registro. Remove o nó tratando os casos de folha, um filho ou dois filhos; retorna 1 quando remove e 0 quando a chave não é encontrada.

`int alturaArvore(ArvoreBinaria *raiz)` recebe a raiz e retorna a altura da árvore, calculada a partir das subárvores esquerda e direita.

`int contaNos(ArvoreBinaria *raiz)` recebe a raiz e retorna a quantidade total de nós da árvore.

`void imprimirArvore(ArvoreBinaria *raiz)` recebe a raiz e percorre a árvore para exibir os registros armazenados.

`void liberaArvore(ArvoreBinaria *raiz)` recebe a raiz e libera recursivamente a memória de todos os nós da árvore.

As funções de validação e leitura também apoiam o programa, como `leChaveSoDigitos(char *destino, int tamanho, char *rotulo)`, que lê CPF ou CEP, e `processaDado(char *dado)`, que valida o dado informado.

No `main.c`, as **mesmas** funções são chamadas para `arvoreCliente` e `arvoreCidade`:

```
1 inserirArvore(&arvoreCliente, criaNoCliente(dadocli));
2 inserirArvore(&arvoreCidade, criaNoCidade(dadocid));
3 buscar(arvoreCliente, cpf);          buscar(arvoreCidade, cep);
4 removerNo(&arvoreCliente, cpf);      removerNo(&arvoreCidade, cep);
5 imprimirArvore(arvoreCliente);      imprimirArvore(arvoreCidade);
```

Funcionou? Sim. O motivo é que todo o algoritmo de árvore binária de busca depende de apenas duas coisas: uma **chave comparável** e os **ponteiros dos filhos**. Ambas ficam no nó genérico, e a obtenção da chave foi empurrada para `pegaChave()`. Como CPF e CEP são strings de dígitos de tamanho fixo, `strcmp` dá a mesma ordem que a ordem numérica.

Isso facilita o desenvolvimento? Facilita, porém:

- *A favor:* um único código de inserção/busca/remoção para manter e testar — um bug corrigido vale para as duas árvores; acrescentar um terceiro tipo custa só um valor novo no `enum`, um campo na `union` e um `case` em `pegaChave()`; menos código duplicado significa menos chance de as duas versões divergirem com o tempo.
- *Contra:* perde-se verificação de tipo em tempo de compilação — nada impede, para o compilador, inserir um nó de cidade na árvore de clientes; o programa aceitaria e a árvore ficaria ordenada por chaves de naturezas diferentes. Além disso, todo nó ocupa o `sizeof` do maior registro.

2.5 Validação dos dados

CPF e CEP são lidos como texto para preservar zeros à esquerda e evitar limitações de tipos numéricos. A função `leChaveSoDigitos` recebe o destino, o tamanho esperado e um rótulo para a entrada. Depois, `processaDado` aplica as regras de validação antes da inserção na árvore.

3 Problemas

Durante o desenvolvimento, foram considerados os seguintes problemas e decisões técnicas:

1. **Duplicação de estruturas:** originalmente seria necessário manter uma árvore para clientes e outra para cidades com funções diferentes. A união e o campo de tipo permitiram um nó genérico e funções compartilhadas.
2. **Leitura de chaves:** CPF e CEP não devem ser lidos como inteiros, pois podem conter zeros à esquerda. A leitura textual também evita overflow para CPFs com 11 dígitos.
3. **Entrada inválida:** a leitura foi separada da validação semântica. Assim, a função de leitura garante a quantidade e o formato dos dígitos, enquanto as funções de validação verificam as regras específicas.
4. **Chaves duplicadas:** a inserção retorna zero quando encontra uma chave já cadastrada. Nesse caso, o nó recém-alocado é liberado para evitar vazamento de memória.
5. **Remoção com dois filhos:** além de copiar as informações do sucessor, é necessário preservar o campo `tipoN`, pois ele identifica qual membro da união deve ser lido.
6. **Desbalanceamento:** a ABB não possui rotações nem balanceamento automático. Por isso, uma sequência ordenada pode transformar a árvore em uma lista encadeada.

4 Resultados

4.1 Equipamento utilizado

Os testes foram realizados no equipamento com as configurações informadas na Tabela 1.

Tabela 1: Configuração do equipamento utilizado nos testes.

Componente	Configuração
Modelo	ASUS TUF
Processador	Core i5 11210H (2.20 GHz)
Memória RAM	16 GB
Armazenamento	512 GB
Placa de Vídeo	RTX 3050 6GB VRAM

4.2 Experimento com 100 CPFs

Foram gerados 100 CPFs distintos e válidos. Os mesmos 10 CPFs, escolhidos nas posições 0, 10, 20 e assim por diante até 90 do vetor ordenado, foram buscados em todas as árvores. O programa contabilizou os nós percorridos em cada busca, calculou a média e registrou a altura de cada árvore.

Tabela 2: Número de nós percorridos para encontrar cada CPF.

CPF buscado	Caso 1	Caso 2	Caso 3	Caso 4
01286898625	1	100	4	10
15680926873	11	90	8	5
25491598100	21	80	8	10
31668836904	31	70	8	11
38584239391	41	60	9	8
49693661109	51	50	1	9
57134192407	61	40	6	6
68638383622	71	30	9	7
77715130508	81	20	9	5
90747286698	91	10	5	5
Média	46,0	55,0	6,7	7,6
Altura da árvore	100	100	12	13
CPF não cadastrado	16	86	7	10

Legenda: Caso 1 corresponde à inserção crescente; Caso 2, à inserção decrescente; Caso 3, ao CPF do meio inserido primeiro e os demais em ordem aleatória; Caso 4, à ordem totalmente aleatória.

5 Discussões

Nos Casos 1 e 2, a ordem de inserção fez a árvore degenerar para uma estrutura semelhante a uma lista. A altura chegou a 100, e o custo da busca passou a ser linear, isto é, $O(n)$. No caso crescente, o primeiro CPF pesquisado foi encontrado na raiz, mas os CPFs seguintes exigiram caminhos cada vez maiores. No caso decrescente, ocorreu o comportamento simétrico.

Os Casos 3 e 4 apresentaram alturas 12 e 13, respectivamente, e médias inferiores a oito nós visitados. Esses resultados mostram o benefício de distribuir os elementos entre os dois lados da raiz. A inserção do elemento central primeiro ajuda a evitar uma raiz extrema, mas não garante, sozinha, uma árvore perfeitamente balanceada, pois os demais elementos continuam sendo inseridos de forma aleatória.

A busca por um CPF ausente terminou quando foi alcançada uma subárvore vazia. Seu custo depende do caminho até esse ponto e, portanto, é limitado pela altura da árvore. Nos cenários ordenados, o custo foi muito superior ao dos cenários aleatórios.

A implementação possui custo médio próximo de $O(\log n)$ quando a árvore permanece razoavelmente equilibrada, mas pode chegar a $O(n)$ no pior caso. Para aplicações que exigem desempenho previsível, uma evolução natural seria utilizar uma árvore AVL ou rubro-negra, que mantém a altura em ordem logarítmica por meio de operações de balanceamento.

6 Conclusão

A implementação mostrou que uma única estrutura genérica pode representar árvores de clientes e cidades sem duplicar as operações fundamentais. A separação da chave por meio da função `pegaChave` tornou o código reutilizável, enquanto a união reduz a quantidade de estruturas necessárias.

O experimento confirmou que a ordem de inserção é determinante para o desempenho de uma ABB não balanceada. Dados inseridos em ordem crescente ou decrescente produziram árvores degeneradas, enquanto a aleatorização manteve alturas menores e buscas mais rápidas. Portanto, a estrutura atende ao objetivo do trabalho, mas seu uso em cenários maiores ou com entradas ordenadas deve considerar uma estratégia de balanceamento.